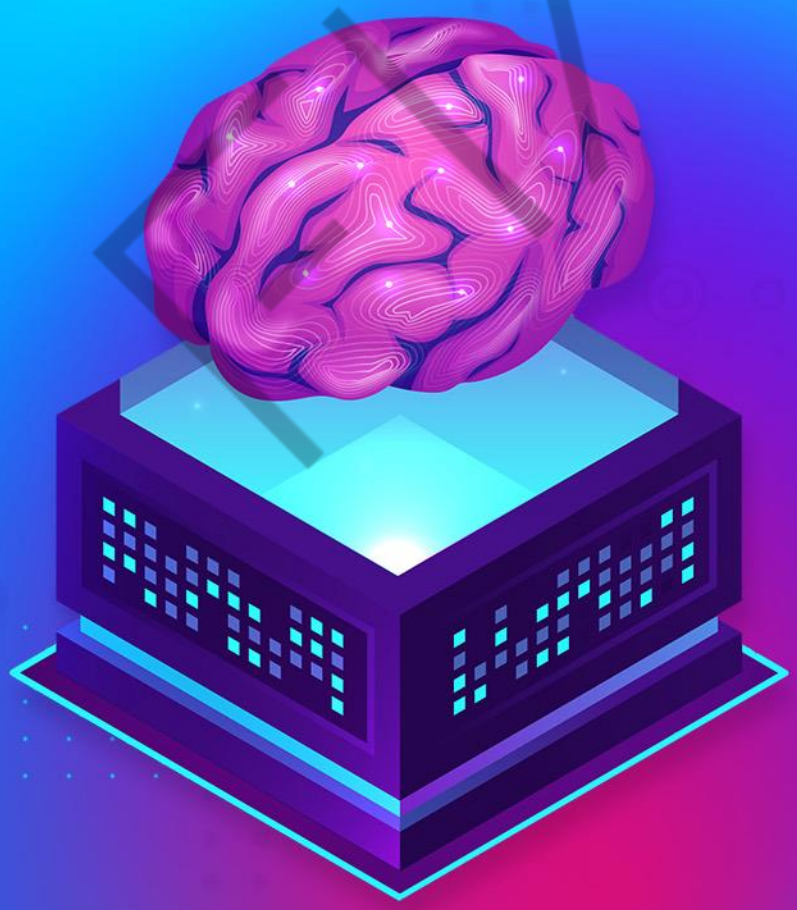
FIAP ON

DATA VIZ & DATA SCIENCE

INTRODUÇÃO ÀS REDES NEURAIS ARTIFICIAIS



17

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Cérebro humano 7
Figura 2 – Representação simplificada do neurônio biológico 8
Figura 3 – Representação de uma rede neural artificial.....9
Figura 4– Comparação do neurônio biológico com o neurônio matemático.....10
Figura 5 – Componentes do Perceptron 13
Figura 6 – Principais funções de ativação: Degrau (a) e Sigmoide (b).....14
Figura 7 – Outras funções de ativação.....15
Figura 8 – Tabela verdade do E LÓGICO (*and*).....18
Figura 9 – Representação do Perceptron que modelará a tabela verdade do E LÓGICO (*and*).....19
Figura 10 – Tipos de flores do “Iris Dataset” 32
Figura 11 – Resultado do código-fonte 6 34
Figura 12 – Resultado do código-fonte 9 35
Figura 13 – Resultado do código-fonte 10 36
Figura 14 – Resultado do código-fonte 11 36
Figura 15 – Resultado do código-fonte 12 37
Figura 16 – Resultado do código-fonte 13 37
Figura 17 – Resultado do código-fonte 14 41

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Resolução da aplicação de um Perceptron simples 17
Quadro 2 – Análise do valor do imóvel pelo Perceptron 18
Quadro 3 – Predição para x1=x2=0 19
Quadro 4 – Processo de atualização dos pesos do Perceptron 21
Quadro 5 – Processo de atualização dos pesos do Perceptron 22
Quadro 6 – Saída da execução do código-fonte 3 30
Quadro 7 – Saída da execução do código-fonte 4 31

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE TABELAS

Tabela 1 – Perceptron modelado para análise de valor de imóveis 17
Tabela 2 – Modelagem do Perceptron considerando W1=W2=0 20
Tabela 3 – Modelagem do Perceptron considerando W1=W2=0,1 21
Tabela 4 – Modelagem do Perceptron considerando W1=W2=0,2 22
Tabela 5 – Modelagem do Perceptron considerando W1=W2=0,5 22

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Importação da biblioteca e criação de variáveis 27
Código-fonte 2 – Código principal da aplicação 28
Código-fonte 3 – Disparando a execução do código principal 29
Código-fonte 4 – Exibindo os pesos finais 30
Código-fonte 5 – Imports iniciais 33
Código-fonte 6 – Importação do dataset e montagem do DataFrame.....33
Código-fonte 7 – Divisão dos dados em treino e teste 34
Código-fonte 8 – Definição e treinamento do Perceptron.....35
Código-fonte 9 – Pesos do Perceptron treinado.....35
Código-fonte 10 – Bias do Perceptron treinado.....35
Código-fonte 11 – Predição no conjunto de teste.....36
Código-fonte 12 – Cálculo da acurácia 36
Código-fonte 13 – Cálculo da matriz de confusão.....37
Código-fonte 14 – Proposta de solução para o desafio.....40

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

INTRODUÇÃO ÀS REDES NEURAIS ARTIFICIAIS.....7
1 CONCEITOS PRELIMINARES 7
1.1 Desvendando o aprendizado humano.....7
1.2 Mergulhando no universo das redes neurais artificiais.....9
2 APLICAÇÕES DE REDES NEURAIS 11
2.1 Quando devemos usar uma rede neural? 11
2.2 Principais casos de uso.....11
3 PERCEPTRON 13
3.1 Definição e componentes.....13
3.2 Funções de ativação 14
3.3 Exemplo de utilização do Perceptron 16
4 TREINAMENTO DO PERCEPTRON 18
4.1 Intuição do processo 18
4.2 Consolidação das etapas de treinamento do Perceptron.....24
4.3 Considerações sobre a taxa de aprendizagem 25
4.3 Considerações finais 26
5 MÃO NA MASSA! PERCEPTRON EM PYTHON.....26
5.1 Programando o ajuste dos pesos.....27
5.2 Utilizando a biblioteca scikit-learn 32
6 HANDS ON: CONSTRUINDO MODELOS PREDITIVOS COM PERCEPTRON ...38
6.1 Seu desafio 38
6.2 Proposta de solução.....39
REFERÊNCIAS.....42
GLOSSÁRIO 43

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

INTRODUÇÃO ÀS REDES NEURAIS ARTIFICIAIS

1 CONCEITOS PRELIMINARES

1.1 Desvendando o aprendizado humano

O aprendizado é a essência da evolução humana, um processo complexo que nos permite adquirir conhecimento e adaptar nosso comportamento ao ambiente em constante mudança. **Mas como exatamente aprendemos?** A resposta para essa pergunta reside nas intrincadas conexões do nosso cérebro.

Nosso **cérebro** (Figura 1) é uma maravilha da natureza, composto por bilhões de neurônios, cada um conectado a milhares de outros. Essas conexões são o alicerce do aprendizado. Quando aprendemos algo, as *sinapses* — as conexões entre os neurônios — são fortalecidas. Esse processo, chamado *plasticidade sináptica*, é a base da nossa capacidade de adquirir novas habilidades e conhecimentos.

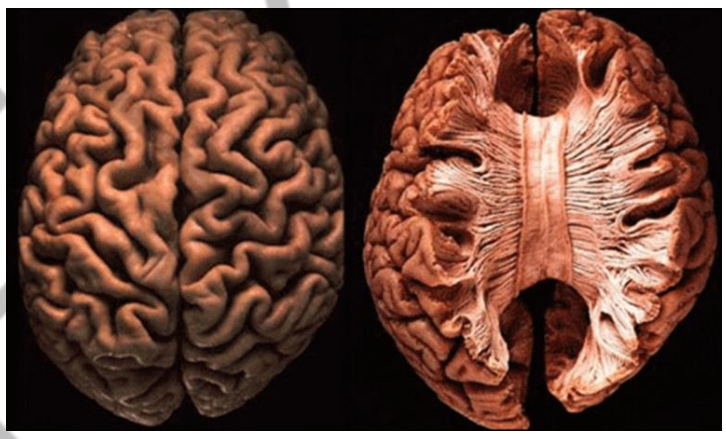


Figura 1 – Cérebro humano
Fonte: Imagem reproduzida de https://blog.portaldomedico.com/anatomia-do-cerebro/

Você sabia?

Conheça alguns fatos curiosos sobre o nosso cérebro:

- O cérebro humano possui cerca de **86 bilhões de neurônios**, que são células especializadas responsáveis pela transmissão de informações no sistema nervoso;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- O cérebro humano representa apenas cerca de **2% do peso corporal total**, mas consome cerca de 20% do oxigênio e da energia do corpo;
- A informação viaja no cérebro a uma velocidade de até **120 metros por segundo**, tornando-o incrivelmente rápido no processamento de informações;
- O cérebro humano é **capaz de gerar cerca de 20 watts de energia** elétrica, o suficiente para alimentar uma lâmpada pequena;
- As conexões neurais no cérebro humano são tão numerosas que, se fossem esticadas, poderiam **envolver a Terra cerca de quatro vezes;**
- O cérebro é incrivelmente adaptável e pode se reorganizar para **compensar danos** (o tal fenômeno da "plasticidade cerebral").

O **aprendizado humano** é um fenômeno multifacetado. Envolve a absorção, compreensão e aplicação de informações, tornando-se um alicerce da nossa existência. Desde os primeiros anos de vida, nosso cérebro é uma esponja ávida, absorvendo experiências e transformando-as em conhecimento. Isso é possível graças a *neurônios interconectados que formam redes complexas*.

Você se lembra das aulas de Biologia? As **redes neurais biológicas** são a inspiração por trás das Redes Neurais Artificiais (RNA). Elas consistem em neurônios interconectados, onde cada neurônio recebe e envia sinais elétricos e químicos (Figura 2). A maneira como essas redes biológicas processam informações, formam associações e se adaptam ao ambiente serviu como modelo para o desenvolvimento de redes neurais artificiais.

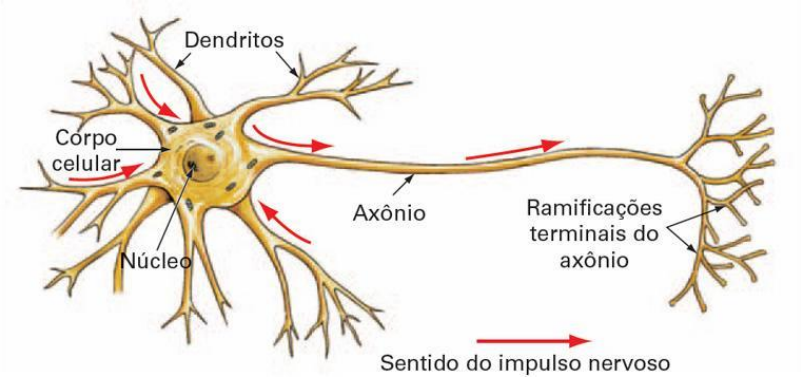


Figura 2 – Representação simplificada do neurônio biológico

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fonte: Imagem reproduzida de https://www.deeplearningbook.com.br/o-neuronio-biologico-e-matematico/

1.2 Mergulhando no universo das redes neurais artificiais

As **redes neurais artificiais** são estruturas de processamento de informações baseadas em algoritmos matemáticos que imitam as redes neurais biológicas. Assim como nossos cérebros aprendem com a experiência, as redes neurais artificiais aprendem a partir dos dados. Elas são compostas por camadas de neurônios artificiais, conectados por pesos que ajustam suas conexões à medida que aprendem (Figura 3).

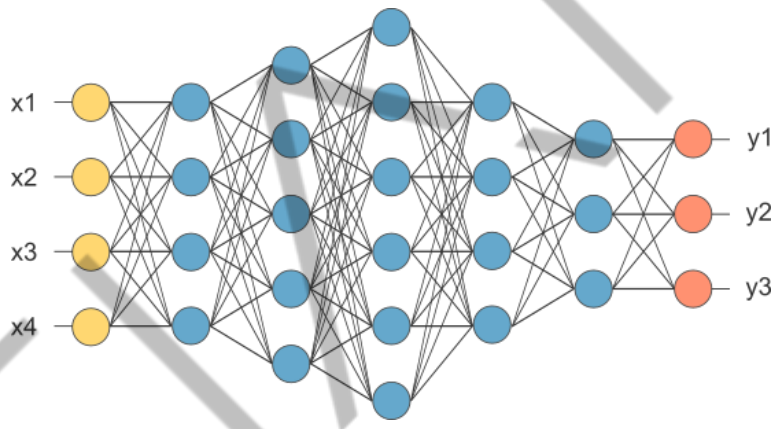


Figura 3 – Representação de uma rede neural artificial
Fonte: Imagem reproduzida de https://www2.decom.ufop.br/imobilis/fundamentos-de-redes-neurais/

A Figura 3 mostra que uma rede neural artificial é composta basicamente por neurônios artificiais organizados em camadas. Temos os seguintes tipos de camadas:

- **Camada de entrada**: neurônios conectados diretamente aos dados;
- **Camadas ocultas**: uma ou várias camadas internas que organizam as combinações dos dados para modelar uma tarefa de aprendizado;
- **Camada de decisão**: a camada com os neurônios que dão a resposta ao problema modelado.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Embora as redes neurais artificiais sejam simplificações das redes neurais biológicas, elas demonstraram notável eficácia em tarefas complexas, como reconhecimento de padrões, processamento de linguagem natural e jogos estratégicos. Essa ponte entre a biologia e a tecnologia é um testemunho do nosso desejo de compreender e replicar os princípios subjacentes ao aprendizado humano. É uma jornada fascinante rumo a máquinas capazes de aprender, evoluir e, quem sabe, compreender como nós, seres humanos, entendemos o mundo ao nosso redor.

Mas a rede neural artificial nem sempre foi como ilustrada na Figura 3. A ideia de uma “rede artificial” é mais moderna e um avanço da ideia básica do “**neurônio artificial**”, também conhecido como “neurônio matemático”, ilustrado na Figura 4, e que conheceremos a fundo logo mais.

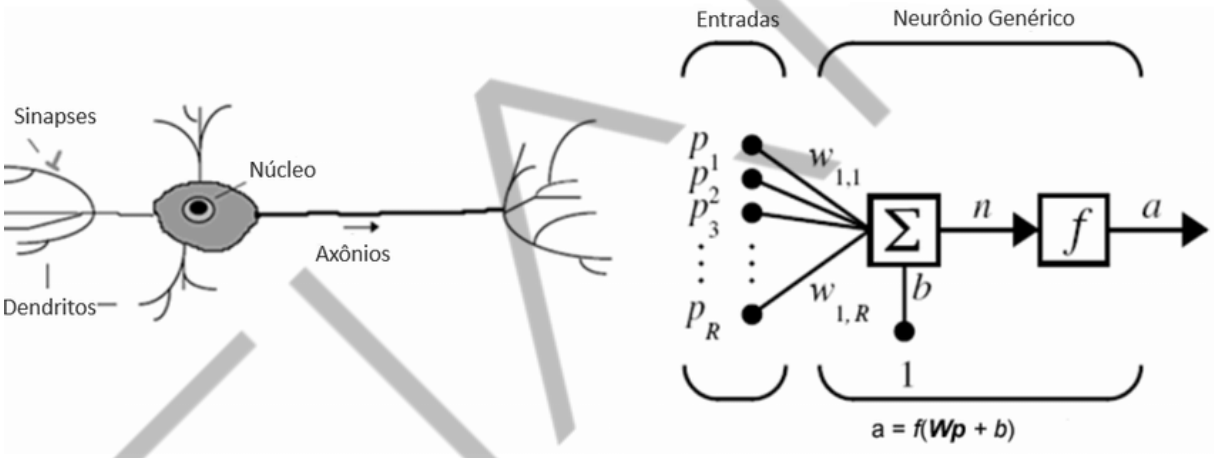


Figura 4– Comparação do neurônio biológico com o neurônio matemático
Fonte: Imagem reproduzida de https://www.researchgate.net/figure/Figura-13-Celula-de-um-neuronio-natural-e-modelo-matematico-respectivo_fig11_332686408

Hoje, as RNAs são uma das ferramentas mais poderosas em ciência de dados e IA, impulsionando avanços em uma ampla gama de aplicações, desde carros autônomos até assistentes de voz e diagnósticos médicos. Seu ressurgimento exemplifica como a perseverança e a inovação podem revolucionar uma área de pesquisa e transformar a maneira como interagimos com a tecnologia.

Detalharemos os componentes da rede neural artificial futuramente. Por enquanto, focaremos nos aspectos básicos e introdutórios, ou seja: no neurônio artificial.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 APLICAÇÕES DE REDES NEURAIS

2.1 Quando devemos usar uma rede neural?

Vimos que as Redes Neurais Artificiais são um subconjunto das técnicas de aprendizado de máquina, as quais abrangem uma infinidade de algoritmos.

Nesse ponto, você deve estar se perguntando: **se as RNAs são um dos possíveis algoritmos de machine learning, quando devo utilizá-las?** A resposta dessa pergunta não é óbvia. Não há como saber, a priori, qual método de aprendizado de máquina melhor se adequará aos dados e/ou ao problema que temos. Portanto, é sempre necessário experimentar as possibilidades.

Por via de regra, podemos considerar que devemos usar RNAs quando:

- Não há um algoritmo que modela o problema de forma mais direta;
- Temos muitos dados à nossa disposição para treinar o modelo;
- Estamos lidando com um problema complexo, sobretudo dados não estruturados (no caso de Deep Learning).

2.2 Principais casos de uso

Atualmente, as aplicações de RNAs se confundem com as aplicações de ponta de IA. Isso se deve à reviravolta das RNAs a partir dos anos 2000 graças ao aumento da capacidade computacional e o desenvolvimento de algoritmos de treinamento mais eficientes. Com isso, as RNAs passaram a ser uma das principais ferramentas técnicas para construção de soluções robustas a problemas complexos.

Mesmo assim, podemos listar uma série de aplicações com sucesso de RNAs “simples”, também chamadas de “RNAs rasas”, tais como:

- **Filtros de Imagem e Reconhecimento de Padrões**: redes simples, como o próprio neurônio matemático, podem ser usadas para detecção

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

de bordas, realce de características e reconhecimento de objetos em imagens, sendo úteis em aplicativos de visão computacional;

- **Previsão Financeira**: RNAs rasas podem ser aplicadas para prever tendências de mercado, preços de ações e flutuações cambiais com base em dados históricos;
- **Sistemas de Recomendação**: as RNAs são usadas em sistemas de recomendação, como os da Netflix e Amazon, para sugerir produtos ou conteúdo com base no histórico e preferências do usuário;
- **Controle de Processos Industriais**: em aplicações industriais, RNAs podem ser usadas para otimizar o controle de processos, como o ajuste de parâmetros em uma linha de produção;
- **Reconhecimento de Escrita Manual**: as RNAs podem ser usadas para identificar e converter escrita manual em texto digital, tornando mais fácil o reconhecimento de caracteres;
- **Detecção de Fraudes**: em setores financeiros, RNAs podem ser usadas para identificar transações suspeitas e atividades fraudulentas com base em padrões de uso do cartão de crédito;
- **Previsão Meteorológica**: as RNAs rasas podem ser aplicadas para prever o clima, analisando dados históricos e condições atuais;
- **Análise de Sentimento em Mídias Sociais**: as RNAs podem ser usadas para determinar a polaridade das opiniões expressas em mídias sociais, ajudando as empresas a avaliar a recepção de seus produtos ou serviços.

Embora essas aplicações não explorem o potencial do aprendizado profundo (Deep Learning), as RNAs simples e multicamadas rasas ainda desempenham um papel significativo em várias tarefas de processamento de informações e são uma base importante para o desenvolvimento de redes mais complexas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3 PERCEPTRON

3.1 Definição e componentes

O Perceptron é um dos blocos fundamentais no mundo das Redes Neurais Artificiais. Embora seja um modelo simples, sua compreensão é crucial para construir redes neurais mais complexas que lidam com tarefas sofisticadas de reconhecimento de padrões, processamento de linguagem natural e muito mais.

Para entendê-lo, vamos começar pela sua definição. O Perceptron é um modelo simples de neurônio artificial, inspirado pelo funcionamento dos neurônios biológicos. Ele foi desenvolvido por Frank Rosenblatt na década de 1950. Os componentes principais do Perceptron estão ilustrados na Figura 5.

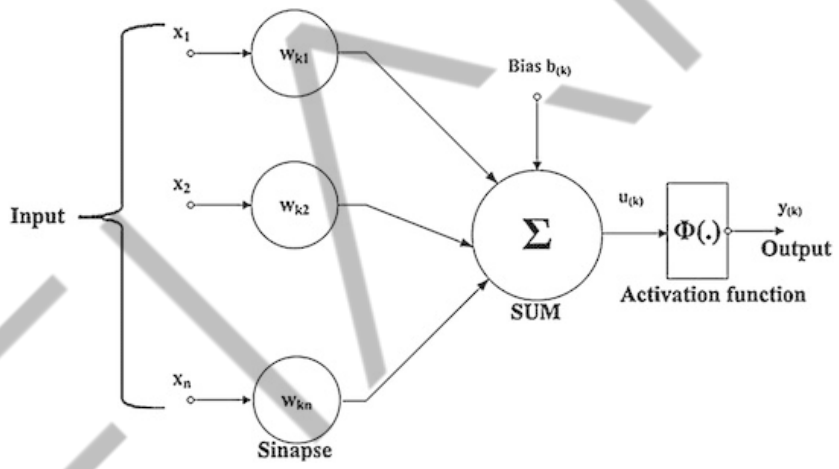


Figura 5 – Componentes do Perceptron
Fonte: Imagem reproduzida de https://www.deeplearningbook.com.br/o-neuronio-biologico-e-matematico/

Em suma, o Perceptron é um algoritmo de **classificação binária** que toma entradas, aplica pesos a essas entradas, soma-as e, em seguida, passa o resultado por uma função de ativação. Essa função de ativação decide se o Perceptron deve "disparar" (produzir uma saída de 1) ou não (produzir uma saída de 0). Matematicamente, a saída do Perceptron é calculada como a soma ponderada das entradas mais um viés (do Inglês "*bias*") (um valor adicional usado para ajustar o limite de ativação).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Devido a sua simplicidade, o Perceptron possui aplicações práticas limitadas às questões de “sim ou não”. Por exemplo, pode ser treinado para distinguir entre números escritos à mão que representam dígitos 0 e 1.

3.2 Funções de ativação

A resposta final do Perceptron é definida pela **função de ativação**. As funções de ativação desempenham um papel crucial no funcionamento do Perceptron. Elas determinam se o neurônio deve ser ativado (produzir uma saída diferente de zero) ou não. Duas das funções de ativação mais comuns são a função degrau (*step*) e a função sigmoide (Figura 6).

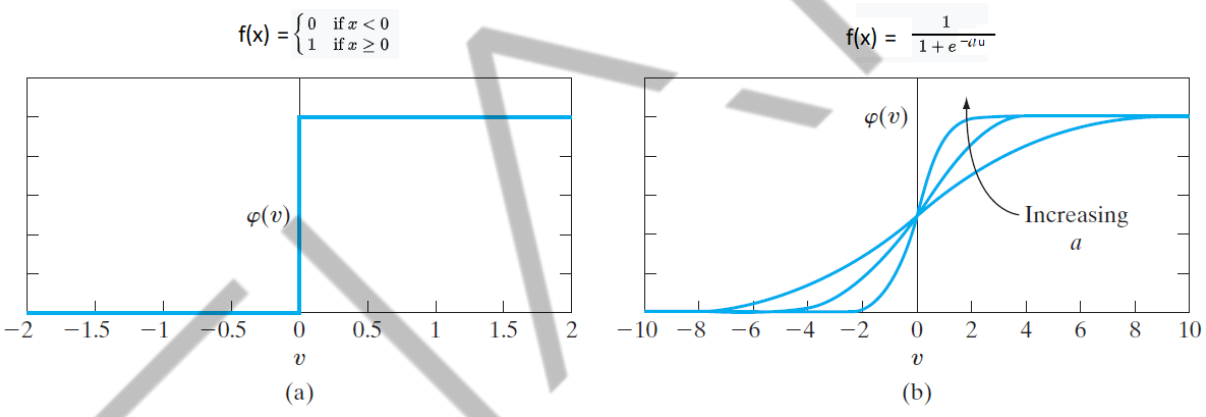


Figura 6 – Principais funções de ativação: Degrau (a) e Sigmoide (b)
Fonte: Imagem adaptada de https://www.researchgate.net/figure/Figura-10-a-Funcao-de-ativacao-degrau-b-funcao-sigmoide-HAYKIN-et-al-2009_fig9_328938085

A **função degrau** é simples: se a soma ponderada das entradas exceder um certo limite, a saída é 1; caso contrário, é 0. A **função sigmoide** produz saídas contínuas entre 0 e 1, o que é útil para problemas de probabilidade.

Na prática, há uma ampla gama de funções de ativação, cada uma com sua curva característica e função de geração. A Figura 7 ilustra algumas delas:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

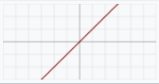
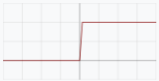
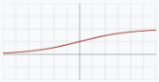
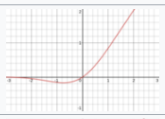
Name	Plot	Function, $g(x)$	Derivative of g, $g'(x)$	Range	Order of continuity
Identity		x	1	$(-\infty, \infty)$	C^∞
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	0	$\{0, 1\}$	C^{-1}
Logistic, sigmoid, or soft step		$\sigma(x) \doteq \frac{1}{1+e^{-x}}$	$g(x)(1-g(x))$	$(0, 1)$	C^∞
Hyperbolic tangent (tanh)		$\tanh(x) \doteq \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - g(x)^2$	$(-1, 1)$	C^∞
Soboleva modified hyperbolic tangent (smht)		$\operatorname{smht}(x) \doteq \frac{e^{ax} - e^{-bx}}{e^{cx} + e^{-dx}}$		$(-1, 1)$	C^∞
Rectified linear unit (ReLU)[8]		$(x)^+ \doteq \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} = \max(0, x) = x\mathbf{1}_{x>0}$	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$	$[0, \infty)$	C^0
Gaussian Error Linear Unit (GELU)[2]		$\frac{1}{2}x\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right) = x\Phi(x)$	$\Phi(x) + x\phi(x)$	$(-0.17\ldots, \infty)$	C^∞
Softplus[9]		$\ln(1 + e^x)$	$\frac{1}{1+e^{-x}}$	$(0, \infty)$	C^∞

Figura 7 – Outras funções de ativação
Fonte: Imagem adaptada de https://en.wikipedia.org/wiki/Activation_function

Podemos detalhar outras além da “degrau” e “sigmoide”:

- **Função linear (ou identidade – *identity*):** é uma das funções de ativação mais simples e diretas usadas em redes neurais. O principal objetivo da função linear é realizar uma operação de identidade, ou seja, ela simplesmente passa os valores de entrada sem modificação para a camada seguinte. Ou seja, a saída da função linear é igual à entrada;
- **ReLU (*Rectified Linear Unit*)**: a função ReLU é uma das mais populares em redes profundas. Ela é definida como f(x) = max(0, x), o que significa que retorna zero para entradas negativas e a própria entrada para valores positivos. O ReLU ajuda a superar o problema de gradientes desaparecidos (veremos o significado disso adiante), acelerando o treinamento e sendo muito eficiente em termos computacionais;
- **Leaky ReLU**: o Leaky ReLU é uma variante do ReLU que permite um pequeno valor para entradas negativas (por exemplo, f(x) = 0.01x

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

quando x < 0). Isso ajuda a mitigar o problema de neurônios "mortos" (onde a saída sempre é zero) em algumas unidades do ReLU;

- **Função Tangente Hiperbólica (Tanh)**: a função tanh produz saídas no intervalo entre -1 e 1. Ela é frequentemente usada em redes neurais, pois é uma alternativa à função sigmoide e pode ajudar a mitigar o problema de gradientes desaparecidos. A tanh é usada principalmente em camadas ocultas.

A importância de ter várias opções de funções de ativação reside na flexibilidade que elas oferecem. **Diferentes problemas podem se beneficiar de diferentes funções de ativação, dependendo das características dos dados e da arquitetura da rede**. Além disso, em redes profundas, a escolha da função de ativação pode afetar o sucesso do treinamento, evitando problemas como gradientes desaparecidos e explosão de gradientes (conheceremos tais problemas e aprenderemos como mitigá-los adiante).

3.3 Exemplo de utilização do Perceptron

Vamos para um **exercício teórico simples**. Dado um Perceptron cujas entradas (x1, x2 e x3) são respectivamente 1, 2 e 3 e os pesos (w1, w2, w3) são respectivamente 0.1, 0.2 e 0.3, calcule a saída desse Perceptron considerando uma função de ativação degrau (limite=0) e bias=0. A resolução é:

Resolução do exercício teórico

```
X = [1, 2, 3]        #Entradas
W = [0.1, 0.2, 0.3] #Pesos
b = 0                #Viés

Se u = W . X + b, então:
u = x1*w1 + x2*w2 + x3*w3 + b
  = 1*0.1 + 2*0.2 + 3*0.3 + 0
  = 0.1 + 0.4 + 0.9 + 0
u = 1.4

Se vamos usar a função degrau, então:
     f(u) = 1, se u≥0; 0, se u<0
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
Como u = 1.4 ≥ 0, então:

f(u) = 1

Portanto, este Perceptron está ativado!
```

Quadro 1 – Resolução da aplicação de um Perceptron simples
Fonte: Elaborado pelo autor (2023)

Agora, vamos colocar o mesmo exercício sob uma perspectiva prática. Imagina que nosso Perceptron deve responder à seguinte questão: *dadas as características de um apartamento, seu preço de venda está acima do mercado?* Para isso, as características serão as ilustradas na Tabela 1:

Variável	Peso	Valor	Descrição
x1	0,3	600.000	Preço (R$)
x2	0,05	130	Metragem (m²)
x3	0,1	4	Número de quartos (unidades)
Limite	N/A	180.000	Limiar para ajuste decisão da função degrau
Bias	N/A	-5000	Valor do viés a ser aplicado

Tabela 1 – Perceptron modelado para análise de valor de imóveis
Fonte: Elaborado pelo autor (2023)

A resolução está no Quadro 2 abaixo:

Resolução do exercício de avaliação de imóvel

```
X = [x1, x2, x3]      #Entradas
X = [600000, 130, 4] #Entradas
W = [0.3, 0.05, 0.1] #Pesos
b = -5000            #Viés

Se u = W . X + b, então:
u = x1*w1 + x2*w2 + x3*w3 + b
  = 600000*0.3 + 130*0.05 + 4*0.1 + (-5000)
  = 180000 + 6,5 + 0,4 - 5000
u = 175006,9

Se vamos usar a função degrau, com limite=180000 então:
     f(u) = 1, se u≥180000; 0, se u<0

Como u = 175006,9 < 180000, então:

f(u) = 0

Portanto, este Perceptron não está ativado! Logo, o imóvel não
está acima do valor de mercado.
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Quadro 2 – Análise do valor do imóvel pelo Perceptron
Fonte: Elaborado pelo autor (2023)

4 TREINAMENTO DO PERCEPTRON

Acabamos de ver duas aplicações do Perceptron utilizando-o para obter respostas de perguntas relacionadas a classificações binárias. Isso só foi possível porque o Perceptron já estava "configurado", com os *pesos* e *bias* informados.

O que aconteceria nos exemplos anteriores caso os pesos e bias não estivessem disponíveis? Nós não conseguiríamos utilizá-los.

A aplicação que simulamos reflete o emprego de um modelo preditivo já desenvolvido para a tomada de decisão. Caso os pesos não fossem dados, teríamos de encontrá-los automaticamente, por meio da análise de exemplos. **Encontrar os pesos e bias é justamente treinar o Perceptron**.

4.1 Intuição do processo

Para exemplificar o processo de treinamento do Perceptron, vamos modelar um problema simples: o "E LÓGICO" (operação *AND* em condicionais). Sabemos que a tabela verdade do "E LÓGICO" é dada pela Figura 8:

A	B	A e B
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

Figura 8 – Tabela verdade do E LÓGICO (*and*)
Fonte: Imagem reproduzida de https://www.treinaweb.com.br/blog/operadores-logicos

O Perceptron que modelará esse problema pode ser representado pela Figura 9, tendo 2 entradas (x1, x2), e 2 pesos a serem descobertos (W1, W2). Para

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

simplificar, consideraremos bias=0 e a função de ativação será: *f(u)* ➔ *f(u)=1 se u=1; do contrário f(u)=0*.

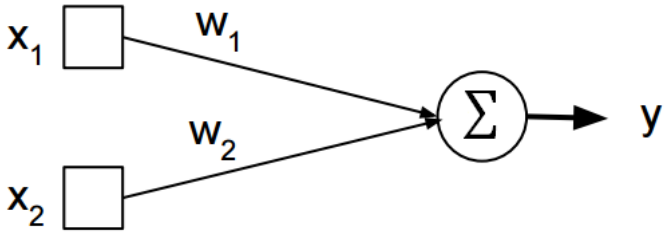


Figura 9 – Representação do Perceptron que modelará a tabela verdade do E LÓGICO (*and*)
Fonte: Elaborado pelo autor (2023)

Como dito, treinar o Perceptron é descobrir os valores de pesos e bias que melhor modelam o problema apresentado como um conjunto de exemplos (ou seja, reduz o erro total das classificações). No nosso caso, sem perda de generalidade, desconsideraremos o bias (bias=0) e focaremos apenas nos pesos.

Nossos exemplos são as 4 linhas da tabela verdade do "E LÓGICO". Precisamos então inicializar os pesos com um valor qualquer. **Vamos começar com W1 = W2 = 0. Será que esse conjunto de pesos modela adequadamente o problema?** Vejamos no Quadro 3 o processo de "predição" quando x1=x2=0:

Treinamento do Perceptron – Predição para x1=x2=0

```
X = [x1, x2]    #Entradas
X = [0, 0]      #Entradas
W = [0.0, 0.0] #Pesos
b = 0           #Viés

Se u = W . X + b, então:
u = x1*w1 + x2*w2 + b
  = 0*0.0 + 0*0.0 + 0
  = 0 + 0 + 0
u = 0

Considerando: f(u) → f(u)=1 se u=1; do contrário f(u)=0

Como u = 0, então:

f(u) = 0

Portanto, este Perceptron não está ativado e a saída predita é 0.
```

Quadro 3 – Predição para x1=x2=0
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Vamos observar na Tabela 2 o resultado completo da reprodução dos passos acima (Quadro 3), mantendo os valores dos pesos, mas alterando as entradas conforme as linhas da tabela verdade:

x1	x2	Saída esperada	f(u)	Saída predita	Status
0	0	0	f(0)	0	Acertou
0	1	0	f(0)	0	Acertou
1	0	0	f(0)	0	Acertou
1	1	1	f(0)	0	Errou

Tabela 2 – Modelagem do Perceptron considerando W1=W2=0
Fonte: Elaborado pelo autor (2023)

A Tabela 2 mostra que o conjunto de pesos W1=W2=0 não modelam o “E LÓGICO”, pois funcionou apenas para 3 das 4 linhas. Ou seja, ainda há erro na classificação.

Assim, temos que calcular os *erros da predição* e *atualizar os pesos*. O **cálculo dos erros** é feito pela soma do erro de cada exemplo, calculado a partir da diferença entre a saída esperada e a saída predita. No ponto acima, o erro total da predição foi 1 (0+0+0+1).

A **atualização dos pesos** segue a fórmula dada pela equação abaixo:

$$\mathbf{W_{novo} = W_{antigo} + (taxa\ de\ aprendizado \times erro \times entrada)}$$

A fórmula mostra que atualizar os pesos é um processo iterativo, sendo que o novo valor do peso a ser explorado depende do valor corrente do peso e de um <u>delta</u>, cujo valor é a multiplicação do valor da entrada associado a esse peso, o erro da predição e a *taxa de aprendizado*.

A **taxa de aprendizado**, também conhecida como "*learning rate*" em Inglês, é um hiperparâmetro crucial em algoritmos de aprendizado de máquina, incluindo redes neurais, que determina o tamanho dos passos que um modelo dá durante o treinamento ao ajustar seus pesos e bias com base no erro calculado. A taxa de aprendizado é um valor escalar positivo que controla a rapidez com que o modelo se adapta aos dados de treinamento. Falaremos mais sobre a taxa de aprendizado adiante. Por enquanto, assuma que é um valor estabelecido antes de iniciarmos o treinamento do Perceptron (ou da Rede Neural).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Ok, vamos focar na última linha da Tabela 2. Quando aplicamos W1=W2=0 nos valores x1=x2=1, há um erro na predição. Portanto, tal erro indica a necessidade de atualizarmos os pesos. Vamos considerar uma taxa de aprendizado igual 0,1. A atualização segue os passos do Quadro 4:

Treinamento do Perceptron – Ajuste dos pesos para x1=x2=1

```
erro = SaídaEsperada - SaídaPredita
erro = 1 - 0
erro = 1

W1novo = W1antigo + (taxa de aprendizado × erro × entrada1)
W1novo = 0 + (0,1 × 1 × 1)
W1novo = 0 + 0,1
W1novo = 0,1

W2novo = W2antigo + (taxa de aprendizado × erro × entrada2)
W2novo = 0 + (0,1 × 1 × 1)
W2novo = 0 + 0,1
W2novo = 0,1

Portanto, na nova iteração, recalcularemos as saídas preditas e o
status da Tabela 2 utilizando W1=W2=0,1.
```

Quadro 4 – Processo de atualização dos pesos do Perceptron
Fonte: Elaborado pelo autor (2023)

Ao utilizarmos W1=W2=0,1 no cálculo das predições de cada linha da tabela verdade do “E LÓGICO”, chegaríamos nos valores da Tabela 3:

x1	x2	Saída esperada	f(u)	Saída predita	Status
0	0	0	f(0)	0	Acertou
0	1	0	f(0,1)	0	Acertou
1	0	0	f(0,1)	0	Acertou
1	1	1	f(0,2)	0	Errou

Tabela 3 – Modelagem do Perceptron considerando W1=W2=0,1
Fonte: Elaborado pelo autor (2023)

Mesmo com os pesos atualizados, continuamos tendo erro na classificação, justamente no caso x1=x2=1. Portanto, devemos atualizar os pesos novamente, seguindo o protocolo previamente introduzido. Ao atualizar os valores do Quadro 4, chegaríamos no processo ilustrado no Quadro 5, abaixo:

Treinamento do Perceptron – Ajuste dos pesos para x1=x2=1

```
erro = SaídaEsperada - SaídaPredita
erro = 1 - 0
erro = 1
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
W1novo = W1antigo + (taxa de aprendizado × erro × entrada1)
W1novo = 0,1 + (0,1 × 1 × 1)
W1novo = 0,1 + 0,1
W1novo = 0,2

W2novo = W2antigo + (taxa de aprendizado × erro × entrada2)
W2novo = 0,1 + (0,1 × 1 × 1)
W2novo = 0,1 + 0,1
W2novo = 0,2

Portanto, na nova iteração, recalcularemos as saídas preditas e o
status da Tabela 2 utilizando W1=W2=0,2.
```

Quadro 5 – Processo de atualização dos pesos do Perceptron
Fonte: Elaborado pelo autor (2023)

Ao utilizarmos W1=W2=0,2 no cálculo das predições de cada linha da tabela verdade do “E LÓGICO”, chegaríamos nos valores da Tabela 4:

x1	x2	Saída esperada	f(u)	Saída predita	Status
0	0	0	f(0)	0	Acertou
0	1	0	f(0,2)	0	Acertou
1	0	0	f(0,2)	0	Acertou
1	1	1	f(0,4)	0	Errou

Tabela 4 – Modelagem do Perceptron considerando W1=W2=0,2
Fonte: Elaborado pelo autor (2023)

Ainda assim haveria erro de classificação! Nesse ponto, você já deve ter entendido o processo. Portanto, antes de continuar, siga com os cálculos à mão e encontre os pesos ideais para este Perceptron.

... (estou aguardando seus cálculos!)...

Certo! Após seus cálculos, você deve ter chegado no valor de W1=W2=0,5, o qual produz os resultados da Tabela 5:

x1	x2	Saída esperada	f(u)	Saída predita	Status
0	0	0	f(0)	0	Acertou
0	1	0	f(0,5)	0	Acertou
1	0	0	f(0,5)	0	Acertou
1	1	1	f(1)	1	Acertou

Tabela 5 – Modelagem do Perceptron considerando W1=W2=0,5
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Pronto! Agora podemos falar que nosso Perceptron está TREINADO, pois encontramos os pesos que modelam o problema apresentado, sem erros.

IMPORTANTE: precisamos fazer algumas considerações sobre o exemplo apresentado! Leia os pontos a seguir com atenção!

Considerações sobre o treinamento do Perceptron que modela o "E LÓGICO":

- Nosso problema possuía apenas 4 exemplos (afinal, a tabela verdade do AND para 2 variáveis possui apenas 4 combinações de valores). Em aplicações reais de machine learning, não faz o menor sentido termos um conjunto de treinamento tão pequeno assim!;
- No nosso exemplo nós não avaliamos o modelo final no conjunto de TESTE porque ele não existe. Em aplicações reais, com datasets reais, isso é fundamental;
- Nós iniciamos os pesos com valor 0 (zero). Na prática, podemos iniciar os pesos com valores aleatórios;
- Nosso modelo parou o treinamento quando o erro da "rede neural" foi igual a zero (todos os exemplos do treino foram corretamente classificados). Na prática, isso é inviável: o TREINO seria dividido em TREINO e VALIDAÇÃO e certamente haveria um percentual de erro na VALIDAÇÃO. **O ponto central do treinamento não é treinar até o erro ser zero, mas até não conseguir melhorar mais os acertos** (ou seja, chegar num erro mínimo do problema, não necessariamente zero);
- O processo de atualização levava a valores iguais para W1 e W2. Isso foi uma coincidência para o exemplo didático apresentado. Na prática, cada peso pode ter valores distintos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4.2 Consolidação das etapas de treinamento do Perceptron

Acabamos de passar pelo processo intuitivo de treinamento do Perceptron. Você deve ter notado que o processo segue uma série de etapas bem definidas. Portanto, é natural pensarmos nesse processo como um algoritmo. Os passos básicos do treinamento do Perceptron são:

1. **Inicialização dos Pesos**: inicialmente, os pesos das entradas são atribuídos a valores aleatórios ou pequenos valores próximos a zero;
2. **Alimentação dos Dados**: um exemplo de treinamento é apresentado ao Perceptron. Os valores das entradas são multiplicados pelos pesos correspondentes, e a soma ponderada é calculada;
3. **Aplicação da Função de Ativação**: a soma ponderada é então passada por uma função de ativação, geralmente uma função degrau. Essa função determina se o Perceptron deve ativar (gerar uma saída de 1) ou não ativar (gerar uma saída de 0) com base na soma ponderada;
4. **Cálculo do Erro**: o resultado da função de ativação é comparado com o rótulo real do exemplo de treinamento. O erro é calculado subtraindo a saída real da saída prevista pelo Perceptron;
5. **Ajuste dos Pesos**: Os pesos das entradas são atualizados com base no erro calculado e na taxa de aprendizado (*learning rate*). A fórmula de atualização é: "$W_{novo} = W_{antigo} + (\text{taxa de aprendizado} \times \text{erro} \times \text{entrada})$". Os pesos são ajustados na direção que reduzirá o erro na próxima iteração;
6. **Iteração**: os passos 2 a 5 são repetidos para todos os exemplos de treinamento até que um critério de parada seja atingido. Isso pode ser:
    a. um número fixo de iterações,
    b. quando o erro atinge um valor mínimo ou,
    c. quando o Perceptron converge e classifica os exemplos corretamente.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

7. **Convergência**: o treinamento do Perceptron converge quando não é mais capaz de cometer erros em um conjunto específico de exemplos de treinamento. Isso significa que ele aprendeu a dividir corretamente os exemplos em suas classes.

4.3 Considerações sobre a taxa de aprendizagem

A escolha apropriada da taxa de aprendizado é crítica para o sucesso do treinamento de um modelo. Vejamos abaixo algumas considerações importantes:

- **Escolha da Taxa de Aprendizado**: a escolha da taxa de aprendizado depende do problema específico e muitas vezes requer ajuste e experimentação. Valores típicos estão na faixa de 0,1 a 0,0001;
- **Taxa de Aprendizado Pequena**: se a taxa de aprendizado for muito pequena, o treinamento pode ser extremamente lento, pois os passos nos ajustes dos pesos são minúsculos. Além disso, pode ser fácil ficar preso em mínimos locais na função de perda (que mede o erro total);
- **Taxa de Aprendizado Grande**: se a taxa de aprendizado for muito grande, o treinamento pode ser instável e divergir, fazendo com que os pesos oscilem e nunca alcancem uma solução ideal;
- **Técnicas de Ajuste**: para lidar com o problema de escolher uma taxa de aprendizado adequada, as técnicas de ajuste automático da taxa de aprendizado, como o "*learning rate decay*" e o uso de otimizadores adaptativos, como o Adam, são comumente empregadas. Veremos essas técnicas adiante;
- **Monitoramento e Ajuste**: durante o treinamento, é importante monitorar o desempenho do modelo no conjunto de validação e ajustar a taxa de aprendizado, se necessário, para otimizar a convergência e evitar problemas como o *overfitting*.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Em resumo, a taxa de aprendizado é um hiperparâmetro crítico que influencia a eficiência e a estabilidade do treinamento de modelos de aprendizado de máquina, incluindo redes neurais. Encontrar o valor adequado para a taxa de aprendizado é muitas vezes uma parte essencial do processo de treinamento de um modelo.

4.3 Considerações finais

Para concluirmos a introdução teórica desse capítulo, é importante destacar que o Perceptron é eficaz apenas na classificação de problemas linearmente separáveis, onde os exemplos de diferentes classes podem ser separados por uma linha (em 2D), um plano (em 3D) ou um hiperplano (em dimensões superiores).

Problemas não linearmente separáveis não podem ser tratados pelo Perceptron. Como resolveremos isso? Veremos futuramente!

5 MÃO NA MASSA! PERCEPTRON EM PYTHON

Agora que conhecemos o Perceptron, está na hora de o utilizarmos na prática para modelarmos problemas de classificação. Primeiramente, implementaremos do zero, em Python, o processo visto no item "**4.1 Intuição do processo**". Depois, veremos como a principal biblioteca de machine learning facilita o treinamento e utilização do Perceptron.

Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

DICA: veja o material HOW TO para acessar o ambiente do Colab.

DICA: uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 3) e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

5.1 Programando o ajuste dos pesos

Primeiramente, precisamos definir a importação da biblioteca NumPy, que facilitará os cálculos matemáticos vistos anteriormente. Também precisamos criar as variáveis relacionadas ao problema:

- nossa entrada será um vetor de vetores (matriz), contendo os 4 pares possíveis de x1 e x2 da tabela verdade do “E LÓGICO”;
- Além disso, cada par x1 e x2 possui uma saída esperada. Assim, teremos outra variável chamada saída, que é um vetor de respostas esperadas pela tabela verdade;
- Como vimos, precisamos de um vetor de pesos, com 2 posições, representando w1 e w2. Vamos inicializá-los com zero;
- Por fim, a taxa de aprendizagem empregada será 0.1.

```
import numpy as np
entradas = np.array([[0, 0],
                     [0, 1],
                     [1, 0],
                     [1, 1]])
saidas = np.array( [0, 0, 0, 1] )
pesos = np.array( [0.0, 0.0] )
taxaAprendizagem = 0.1
```

Código-fonte 1 – Importação da biblioteca e criação de variáveis
Fonte: Elaborado pelo autor (2023)

A execução da célula abaixo não produz saída alguma. Agora, criaremos outra célula na qual definiremos as 3 funções imprescindíveis para o processo de treinamento do Perceptro: a função de ativação, a função que calcula as saídas e a função de treinamento:

```
def funcao_ativacao(soma):
  if soma>=1:
    return 1
  else:
    return 0

# Veja a documentação da função "dot" do numpy neste link:
# https://numpy.org/doc/stable/reference/generated/numpy.dot.html
def calcula_saida(exemplo):
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
  s = exemplo.dot(pesos)
  return funcao_ativacao(s)

def treinar():
  erroTotal = 1

  while erroTotal != 0:
    erroTotal = 0
    for i in range(len(saidas)):
      saidaCalculada = calcula_saida( entradas[i] )
      erro = saidas[i] - saidaCalculada
      erroTotal += erro    # erroTotal = erroTotal + erro
      print(f"\tDado: {i}")
      for j in range(len(pesos)):
        pesos[j] = pesos[j] + (taxaAprendizagem * entradas[i][j] *
erro)
        print(f"\t\tPeso atualizado {j}: {pesos[j]}")
    print(f"---> Total de erros: {erroTotal}\n")
```

Código-fonte 2 – Código principal da aplicação
Fonte: Elaborado pelo autor (2023)

Conforme mencionado no item 4.1, a função de ativação do problema recebe a soma (ponderada) das entradas e verifica se é maior ou igual a 1. Se for, o Perceptron está ativado (retorna 1), do contrário, não está (retorna 0).

Devido à forma como organizamos as entradas e os pesos (vetores), podemos nos beneficiar das facilidades de utilização da biblioteca NumPy para calcularmos o produto escalar de 2 vetores, por meio do método “dot”. Vale relembrar que o “produto escalar” de 2 vetores é o resultado da soma da multiplicação dos elementos de cada vetor. Se considerarmos que as entradas precisam ser multiplicadas pelos pesos correntes para produzir a “soma ponderada”, temos exatamente o que precisamos.

Tal “soma” passa pela função de ativação e daí temos o resultado da ativação ou não do Perceptron naquela iteração do algoritmo.

Agora falta iterar o algoritmo de treinamento. Para efeitos de simplificação, treinaremos enquanto há erro de classificação (while), sendo que para cada exemplo, nós calculamos a saída e comparamos com a saída esperada. Em caso de erro, atualizamos o contador e passamos por todos os pesos atualizando-os seguindo a fórmula padrão.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Os “prints” não são obrigatórios, mas nos ajudam a entender as iterações e as saídas obtidas.

A execução da célula acima também não produz saída. Mas podemos disparar o processo por meio do comando:

```
treinar()
```

Código-fonte 3 – Disparando a execução do código principal
Fonte: Elaborado pelo autor (2023)

O código-fonte acima produz a saída ilustrada no Quadro 6 abaixo:

```
Dado: 0
            Peso atualizado 0: 0.0
            Peso atualizado 1: 0.0
      Dado: 1
            Peso atualizado 0: 0.0
            Peso atualizado 1: 0.0
      Dado: 2
            Peso atualizado 0: 0.0
            Peso atualizado 1: 0.0
      Dado: 3
            Peso atualizado 0: 0.1
            Peso atualizado 1: 0.1
---> Total de erros: 1

      Dado: 0
            Peso atualizado 0: 0.1
            Peso atualizado 1: 0.1
      Dado: 1
            Peso atualizado 0: 0.1
            Peso atualizado 1: 0.1
      Dado: 2
            Peso atualizado 0: 0.1
            Peso atualizado 1: 0.1
      Dado: 3
            Peso atualizado 0: 0.2
            Peso atualizado 1: 0.2
---> Total de erros: 1

      Dado: 0
            Peso atualizado 0: 0.2
            Peso atualizado 1: 0.2
      Dado: 1
            Peso atualizado 0: 0.2
            Peso atualizado 1: 0.2
      Dado: 2
            Peso atualizado 0: 0.2
            Peso atualizado 1: 0.2
      Dado: 3
            Peso atualizado 0: 0.30000000000000004
            Peso atualizado 1: 0.30000000000000004
---> Total de erros: 1
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
      Dado: 0
            Peso atualizado 0: 0.30000000000000004
            Peso atualizado 1: 0.30000000000000004
      Dado: 1
            Peso atualizado 0: 0.30000000000000004
            Peso atualizado 1: 0.30000000000000004
      Dado: 2
            Peso atualizado 0: 0.30000000000000004
            Peso atualizado 1: 0.30000000000000004
      Dado: 3
            Peso atualizado 0: 0.4
            Peso atualizado 1: 0.4
---> Total de erros: 1

      Dado: 0
            Peso atualizado 0: 0.4
            Peso atualizado 1: 0.4
      Dado: 1
            Peso atualizado 0: 0.4
            Peso atualizado 1: 0.4
      Dado: 2
            Peso atualizado 0: 0.4
            Peso atualizado 1: 0.4
      Dado: 3
            Peso atualizado 0: 0.5
            Peso atualizado 1: 0.5
---> Total de erros: 1

      Dado: 0
            Peso atualizado 0: 0.5
            Peso atualizado 1: 0.5
      Dado: 1
            Peso atualizado 0: 0.5
            Peso atualizado 1: 0.5
      Dado: 2
            Peso atualizado 0: 0.5
            Peso atualizado 1: 0.5
      Dado: 3
            Peso atualizado 0: 0.5
            Peso atualizado 1: 0.5
---> Total de erros: 0
```

Quadro 6 – Saída da execução do código-fonte 3
Fonte: Elaborado pelo autor (2023)

Conforme esperado, após os cálculos iniciais, precisamos de 5 iterações para convergir os pesos até os valores que modelam o problema. Para vermos os pesos finais, basta fazer o seguinte comando:

```
pesos
```

Código-fonte 4 – Exibindo os pesos finais
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O qual produzirá a seguinte saída:

```
array([0.5, 0.5])
```

Quadro 7 – Saída da execução do código-fonte 4
Fonte: Elaborado pelo autor (2023)

Detalhe: o valor “estranho” dos pesos como “`0.30000000000000004`” ilustram o comportamento de Python que pode apresentar falhas de arredondamento, mas o erro é tão pequeno que pode ser desconsiderado, aproximando para “`0.3`”.

SUA VEZ!

Execute novamente os comandos acima, alterando os valores dos pesos iniciais e da taxa de aprendizagem, observando o impacto no treinamento da rede.

Note que eventualmente você pode chegar em uma configuração que a rede não converge para os pesos adequados (ou seja, o laço *while* continua a rodar infinitamente. Se isso ocorrer, pare a execução no botão STOP (*interrupt the kernel*) ou vá em Kernel > Restart.

No Colab, a interrupção do código em loop infinito é por "Ambiente de execução" > "Interromper execução".

O que esperamos?

- Outros valores iniciais dos pesos podem levar à convergência do algoritmo;

- Outros pares de valores para w1 e w2 podem modelar perfeitamente o problema;

- Uma taxa de aprendizagem baixa pode demorar mais para convergir o problema;

- Uma taxa de aprendizagem alta pode convergir mais rápido ou até mesmo gerar uma execução infinita.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

5.2 Utilizando a biblioteca scikit-learn

O item anterior mostrou na prática como o processo de treinamento do Perceptron acontece. Felizmente, não precisamos nos preocupar com a implementação do zero do algoritmo de treinamento do Perceptron.

A biblioteca scikit-learn nos presenteia com uma implementação desse algoritmo seguindo as mesmas interfaces das demais estratégias de machine learning já estudadas. Vejamos a seguir!

DICA: Para essa parte do código, utilizaremos um dataset já salvo na memória do scikit-learn, por isso, não há um arquivo CSV associado!

Vamos utilizar a base “Iris” já persistida no scikit-learn. Essa base é um conjunto de dados muito conhecido na área de aprendizado de máquina e classificação. Ele contém informações sobre três espécies diferentes de flores íris: setosa, versicolor e virginica. Cada amostra da base de dados inclui quatro atributos (características): comprimento e largura da sépala e comprimento e largura da pétala, todos medidos em centímetros. O objetivo típico, ao trabalhar com o conjunto de dados Iris, é criar um modelo de aprendizado de máquina que possa classificar corretamente a espécie de uma flor íris com base em suas características.

Para efeitos de curiosidade, a Figura 10 ilustra os tipos de flores desse dataset.

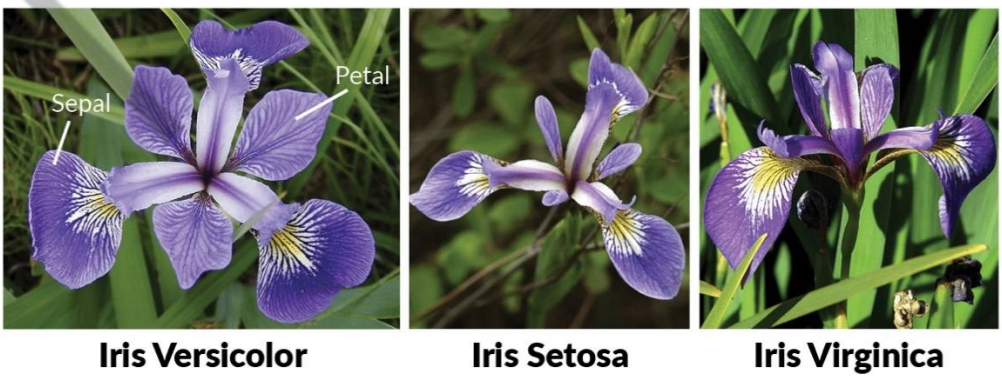


Figura 10 – Tipos de flores do “Iris Dataset”
Fonte: Figura reproduzida de http://www.lac.inpe.br/~rafael.santos/Docs/CAP394/WholeStory-Iris.html

Vamos começar com os *imports*:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Apenas para evitar warnings
import warnings
warnings.filterwarnings('ignore')

# Bibliotecas de Data Science
import pandas as pd

# Separação dos dados
from sklearn.model_selection import train_test_split

# Modelo a ser utilizados
from sklearn.linear_model import Perceptron

# Métricas de avaliação
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import accuracy_score
```

Código-fonte 5 – Imports iniciais
Fonte: Elaborado pelo autor (2023)

A célula acima não produz saídas. Agora, vamos puxar o dataset da memória do sklearn e montar um DataFrame Pandas com os dados. Cada variável terá uma coluna com seu nome e a última coluna, chamada “class”, será o label de cada exemplo:

```
# Lendo o montando o DataFrame com o dataset
from sklearn.datasets import load_iris
iris = load_iris()

# Transformando em Dataframe Pandas:
#   - os dados são iris.data
#   - as colunas são os nomes das características
data = pd.DataFrame( iris.data, columns=iris.feature_names )
data["class"] = iris.target
data
```

Código-fonte 6 – Importação do dataset e montagem do DataFrame
Fonte: Elaborado pelo autor (2023)

Ao executar o código acima, você verá o DataFrame ilustrado na Figura 11, com 150 linhas (exemplos) e 5 colunas (4 variáveis + 1 label).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

Figura 11 – Resultado do código-fonte 6
Fonte: Elaborado pelo autor (2023)

Novamente, **como se trata de um dataset fortemente empregado para ensino de machine learning, ele já se encontra muito bem formatado: não há valores faltantes, dados duplicados ou qualquer outro "problema grave" que precisa ser sanado**. Todas as variáveis são expressas na mesma medida (centímetros) e seus valores são próximos, dispensando padronização ou normalização. Portanto, podemos pular as etapas de limpeza e exploração. Reforçamos que tais etapas são essenciais ao trabalharmos com datasets reais!

Podemos seguir diretamente na divisão dos dados em treino e teste, e aqui separaremos 20% dos dados para teste. Usaremos o "`random_state=42`" para efeitos de reprodutibilidade:

```
# Divisão em TREINO e TESTE
X = data.drop(columns=["class"]) # variáveis
y = data["class"]                # label

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Código-fonte 7 – Divisão dos dados em treino e teste
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Novamente, o código acima não produz saídas. Então, avancemos para a definição e treinamento do Perceptron:

```
# Cria um perceptron e o treina (com o fit())
p = Perceptron( random_state=42 )
p.fit(X_train, y_train)
```

Código-fonte 8 – Definição e treinamento do Perceptron
Fonte: Elaborado pelo autor (2023)

O código-fonte 8 reforça o padrão de interface da biblioteca scikit-learn. Nós definimos o construtor do algoritmo de machine learning pelo nome da classe (importada anteriormente) e configuramos sua utilização. Aqui, a única configuração é indicar a semente aleatória para garantir reprodutibilidade do código. Pense: onde o fator aleatório aparece na teoria do Perceptron? Justamente na escolha dos pesos iniciais! Uma vez com o construtor pronto, invocamos o algoritmo de treinamento do Perceptron por meio do método “fit()”.

Ao encerrar a execução dessa célula, temos o Perceptron treinado, podendo observar seus **pesos**:

```
# Estes são os pesos já treinados, sendo 3 classes (linhas) e 4
colunas (features)
p.coef_
```

Código-fonte 9 – Pesos do Perceptron treinado
Fonte: Elaborado pelo autor (2023)

```
array([[  3. ,   7.9, -12.5,  -5.2],
       [ 15.9, -33.3,   6.1, -24.9],
       [-27.6, -24.9,  27.8,  27.4]])
```

Figura 12 – Resultado do código-fonte 9
Fonte: Elaborado pelo autor (2023)

E também seus ***bias***:

```
# Estes são os bias do perceptron
p.intercept_
```

Código-fonte 10 – Bias do Perceptron treinado
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
array([  2.,  14., -13.])
```

Figura 13 – Resultado do código-fonte 10
Fonte: Elaborado pelo autor (2023)

O exemplo mostrou conjuntos de 4 pesos, um para cada variável, mas por que apresentou 3 conjuntos de pesos e bias? Isso se deve porque temos 3 classes. O Perceptron é um modelo simples utilizado para classificação binária, promovendo uma separação linear das classes. Para cada classe, é como se ele separasse “aquela classe” contra “as outras classes”. Portanto, temos 3 “retas” de separação, cada uma modelando a importância de cada variável (pesos) e o fator constante a ser somado (os bias). Imagine se tivéssemos que controlar o treinamento de cada um desses conjuntos diretamente, como fizemos no item “5.1”? Felizmente, as bibliotecas de machine learning nos facilitam muito a programação!

Nesse ponto, podemos utilizar o Perceptron treinado (com seus pesos e bias definidos) para prever a classificação de outros exemplos:

```
# Salva as predições do TESTE na variável y_pred
y_pred = p.predict(X_test)
y_pred
```

Código-fonte 11 – Predição no conjunto de teste
Fonte: Elaborado pelo autor (2023)

```
array([1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1,
       0, 1, 1, 1, 1, 1, 0, 0])
```

Figura 14 – Resultado do código-fonte 11
Fonte: Elaborado pelo autor (2023)

E uma vez com as classes previstas em mãos, podemos calcular as métricas desse modelo. Começaremos pela **acurácia**:

```
# Calcula a acurácia deste perceptron, por meio dos valores preditos
# para o teste e os valores reais
accuracy_score(y_pred, y_test)
```

Código-fonte 12 – Cálculo da acurácia
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
0.6333333333333333
```

Figura 15 – Resultado do código-fonte 12
Fonte: Elaborado pelo autor (2023)

Que claramente não está boa! Então, vejamos onde há potencial erro de classificação, por meio da **matriz de confusão**:

```
# Matriz de confusão do perceptron
from sklearn.metrics import confusion_matrix

def matriz_confusao(teste_labels, teste_preds, labels):
  cm = confusion_matrix(teste_labels, teste_preds, labels=labels)
  disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                                display_labels=labels)
  disp.plot(cmap="Blues")

matriz_confusao(y_test, y_pred, p.classes_)
```

Código-fonte 13 – Cálculo da matriz de confusão
Fonte: Elaborado pelo autor (2023)

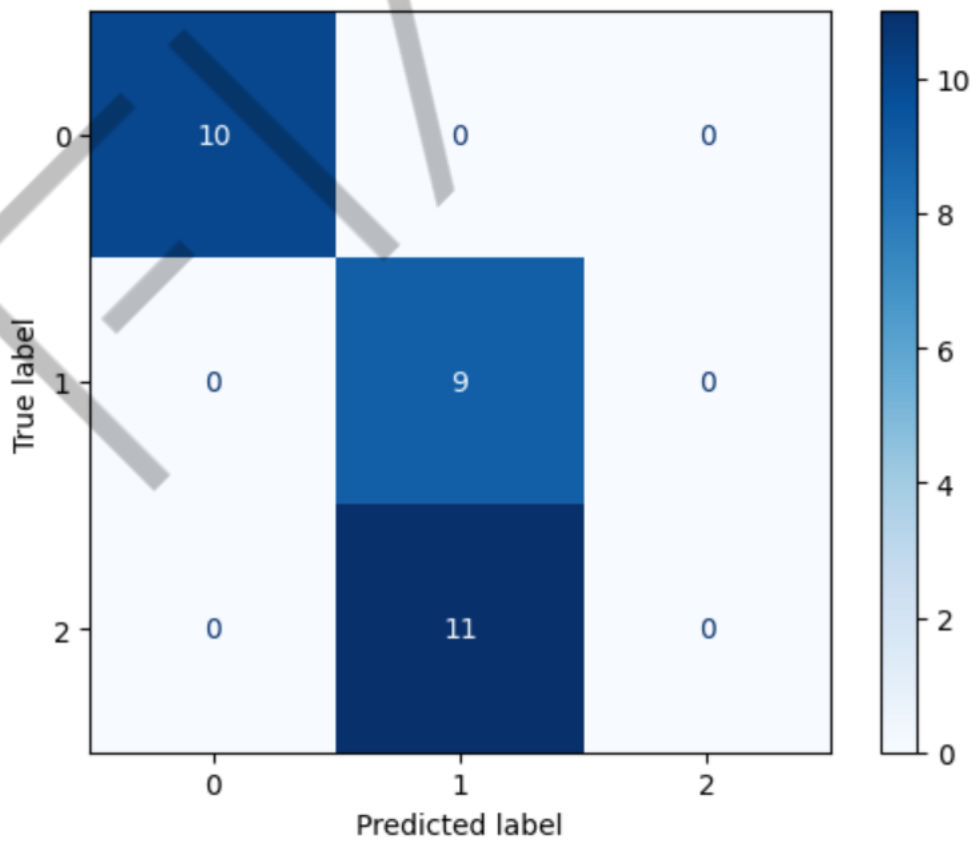


Figura 16 – Resultado do código-fonte 13
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Pronto! A acurácia de ~63% mostra que nosso modelo preditivo de tipo de flor íris não está performando muito bem, e a matriz de confusão da Figura 16 revela que as classes 0 e 1 estão sendo facilmente reconhecidas, mas a classe 2 está sendo erroneamente classificada como 1.

Essa distorção indica que o problema pode ser mais complexo do que esperávamos, sugerindo que ele não seja um problema linearmente separável. Portanto, precisamos de modelos mais robustos, como veremos futuramente!

6 HANDS ON: CONSTRUINDO MODELOS PREDITIVOS COM PERCEPTRON

6.1 Seu desafio

Agora é a sua vez de criar modelos de classificação binária com o Perceptron. Utilizando como base os códigos vistos no material, e a indicação do dataset, sua tarefa será criar um modelo de identificação se um nódulo no seio é um tumor maligno ou benigno.

Para isso, vamos utilizar uma base mundialmente famosa, chamada "*Breast Cancer Wisconsin (Diagnostic) Dataset*":

- **Descrição**: esse conjunto de dados contém informações sobre características extraídas de imagens digitalizadas de massas mamárias. O objetivo é prever se uma massa é benigna ou maligna (diagnóstico de câncer de mama);
- **Onde encontrar**: você pode encontrar esse conjunto de dados no repositório de dados *UCI Machine Learning Repository*, mas já facilitamos o trabalho disponibilizando os dados no arquivo de apoio "breast_cancer.csv";
- **Dicas**:
    - Procure na internet o significado das variáveis!;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Explore o dataset e identifique quais variáveis utilizar, quais deseja descartar, se há relação entre as variáveis, qual coluna é o label, entre outras questões relevantes no processo de ciência de dados;
- Faça vários modelos, usando mais ou menos features disponíveis e compare os resultados. Qual modelo foi o melhor?

6.2 Proposta de solução

Abaixo; temos uma proposta de solução para o desafio apresentado. Essa é apenas uma forma de resolver o problema, utilizando todas as variáveis disponíveis. Seus modelos podem ter resultados distintos dependendo dos tratamentos feitos nos dados e quais features foram empregadas.

O código também assume que o notebook possui a definição da função "`matriz_confusao()`", como fizemos anteriormente.

Aqui, a padronização das features é **mandatória** devido às diferentes escalas entre cada uma delas.

```
# Importar bibliotecas e carregar os dados
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron

# Carregar o conjunto de dados Breast Cancer Wisconsin
df = pd.read_csv("breast_cancer.csv")

# Divisão em DADO e LABEL
X = df.drop(columns=["diagnosis"]) # variáveis
y = df["diagnosis"]                # label

# Divisão em TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Padronizar os recursos
sc = StandardScaler()
sc.fit(X_train)
X_train_std = sc.transform(X_train)
X_test_std = sc.transform(X_test)

# Criar e treinar o Perceptron
perceptron = Perceptron(max_iter=1000, random_state=42)
perceptron.fit(X_train_std, y_train)

# Salva as predições do TESTE na variável y_pred
y_pred = perceptron.predict(X_test_std)
y_pred

# Avaliar o modelo (alunos devem completar esta parte)
display("ACC: " + str(accuracy_score(y_pred, y_test)))

matriz_confusao(y_test, y_pred, perceptron.classes_)
```

Código-fonte 14 – Proposta de solução para o desafio
Fonte: Elaborado pelo autor

O código-fonte 14 produz a saída da Figura 17. Nela, vemos que a acurácia está boa (~98%) e que há um pequeno fator de confusão entre as classes B (benigno) e M (maligno).

O Perceptron proposto errou apenas 3 exemplos, porém, empregou todas as variáveis, tornando-o complexo para uso real. Você conseguiu resultados semelhantes com menos variáveis?

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

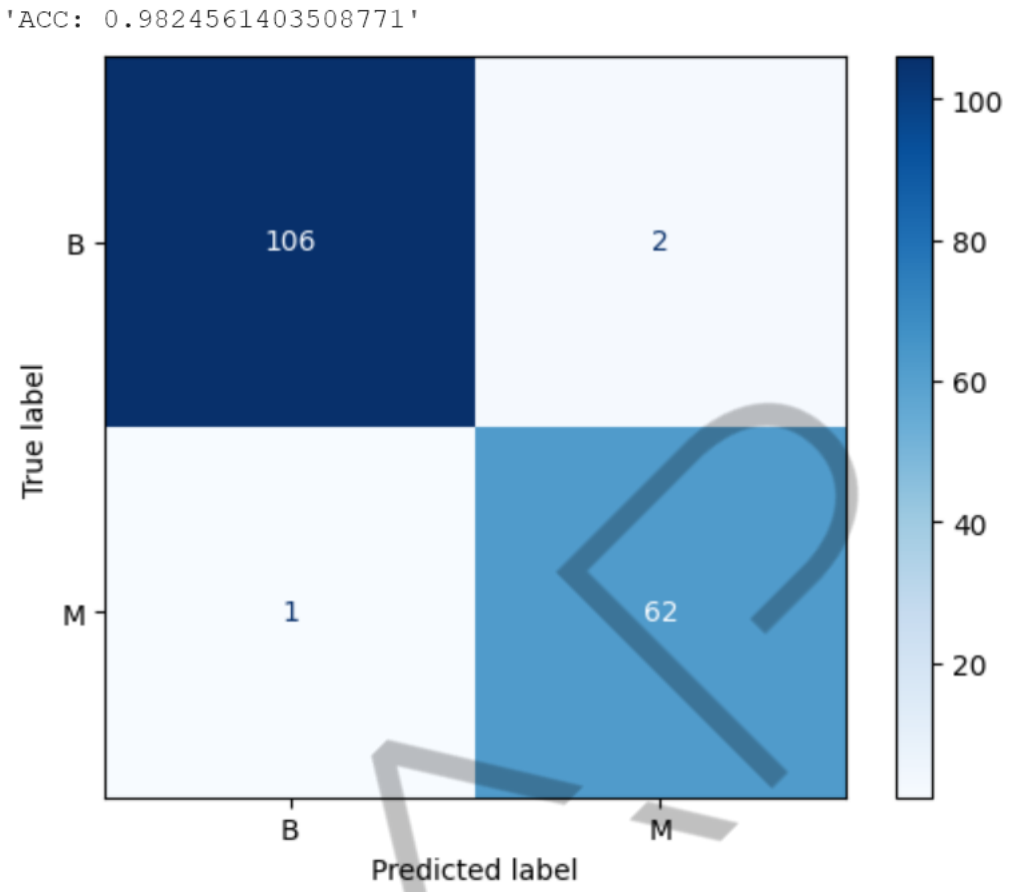


Figura 17 – Resultado do código-fonte 14
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

BLOCK, Hans-Dieter. **The perceptron:** A model for brain functioning. i. Reviews of Modern Physics, 1962, 34.1: 123.

ROSENBLATT, F.; PAPERT, S. **Perceptron**. April, 2021.

VON ZUBEN, Fernando J.; DE CASTRO, Leandro N. **Redes neurais.** Notas de aula, FEEC, Universidade Estadual de Campinas, 2003.

HAYKIN, Simon. **Redes neurais:** princípios e prática. Bookman Editora, 2008.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

GLOSSÁRIO

Termo	**Explicação**
RNA	Rede Neural Artificial
RNAs	Redes Neurais Artificiais
ReLU	Rectified Linear Unit
Tanh	Tangente Hiperbólica

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com